

Technical Documentation

AI Shopping Agent

Frontend Deployment:

<https://ai-fashion-agent-ecru.vercel.app/>

Backend Deployment:

<https://ai-fashion-agent.onrender.com>

GitHub Repository:

<https://github.com/Deepti25sahu/ai-fashion-agent>

Deepti Kumari Sahu

Contents

1	Introduction	3
2	System Overview	3
3	High-Level Architecture	3
4	Frontend Architecture	3
4.1	Framework	3
5	Frontend Components	4
5.1	Navbar	4
5.2	SearchBar	4
5.3	ProductCard	4
5.4	OutfitSuggestions	4
5.5	WishlistDrawer	4
5.6	Footer	4
6	State Management	5
6.1	Main States	5
7	Backend Architecture	5
7.1	Framework	5
7.2	API Endpoint	5
8	AI Agent System	5
8.1	Intent Agent	6
8.2	Search Agent	6
8.3	Recommendation Agent	6
8.4	Ranking Agent	6
8.5	Outfit Agent	6
9	Product Aggregation	6
10	AI Features	7
10.1	Follow-Up Questions	7
10.2	Recommendation Reasoning	7
10.3	Tradeoff Analysis	7
11	Voice Search	7
12	Local Storage	7
13	Deployment Architecture	8
13.1	Frontend Deployment	8
13.2	Backend Deployment	8
14	Deployment Links	8

15 Security Measures	8
16 Future Improvements	9
17 Conclusion	9

1 Introduction

This document explains the technical architecture, system design, implementation details, backend APIs, frontend components, deployment workflow, and AI pipeline used in the AI Fashion Shopping Agent project.

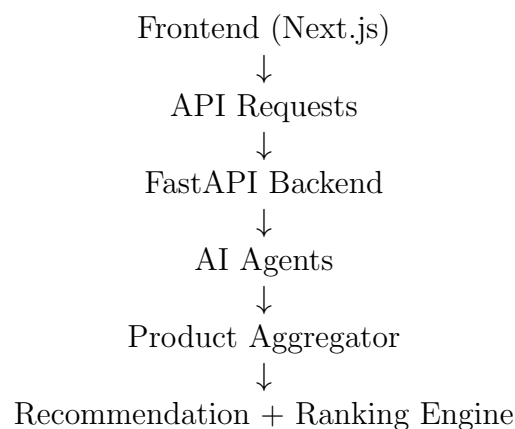
The platform is designed as a conversational AI-powered ecommerce assistant that helps users discover fashion products intelligently.

2 System Overview

The system follows a client-server architecture consisting of:

- Frontend Application (Next.js)
- Backend API Server (FastAPI)
- AI Agent Layer
- Product Aggregation System
- Recommendation Engine
- Deployment Infrastructure

3 High-Level Architecture



4 Frontend Architecture

4.1 Framework

The frontend is built using:

- Next.js 14 App Router
- TypeScript
- Tailwind CSS

- Framer Motion
- Lucide Icons

5 Frontend Components

5.1 Navbar

Responsible for:

- Branding
- Navigation
- Theme controls

5.2 SearchBar

Handles:

- User input
- Search requests
- Voice input integration

5.3 ProductCard

Displays:

- Product image
- Price
- Rating
- Buy button
- Wishlist button
- AI recommendation reasoning

5.4 OutfitSuggestions

Displays AI-generated outfit recommendation chips.

5.5 WishlistDrawer

Stores wishlist products using browser localStorage.

5.6 Footer

Displays branding and informational links.

6 State Management

Frontend state management uses React Hooks.

6.1 Main States

State Variable	Purpose
message	User search query
products	Product list
wishlist	Saved wishlist items
compareProducts	Product comparison data
loading	Loading state
darkMode	Theme toggle
followUp	AI follow-up question

7 Backend Architecture

7.1 Framework

Backend is implemented using:

- FastAPI
- Python
- REST APIs

7.2 API Endpoint

POST /chat

Request Example:

```
{  
  "message": "black dress for farewell"  
}
```

Response Example:

```
{  
  "reply": "...",  
  "products": [...],  
  "outfit": [...],  
  "follow_up": "What occasion is this for?"  
}
```

8 AI Agent System

The backend follows a multi-agent architecture.

8.1 Intent Agent

Detects user intent from natural language.

Example:

- casual fashion
- party wear
- formal outfit

8.2 Search Agent

Generates optimized search queries.

8.3 Recommendation Agent

Filters and selects relevant products.

8.4 Ranking Agent

Ranks products using:

- relevance
- ratings
- style match
- popularity

8.5 Outfit Agent

Suggests matching fashion items dynamically.

9 Product Aggregation

The system aggregates fashion products from external ecommerce sources using APIs.

Each product contains:

- Product Name
- Product Image
- Price
- Rating
- Source Website
- Purchase Link

10 AI Features

10.1 Follow-Up Questions

The system asks intelligent follow-up questions.

Examples:

- What occasion is this for?
- Casual or formal?
- Preferred color?

10.2 Recommendation Reasoning

Each recommendation includes reasoning.

Example: *Recommended because it matches your elegant fashion search.*

10.3 Tradeoff Analysis

The system explains:

- premium quality vs budget
- affordability vs luxury

11 Voice Search

Voice search uses browser speech recognition APIs.

Flow:

1. User clicks microphone button
2. Browser captures speech
3. Transcript converts to search query
4. Query sent to backend API

12 Local Storage

Wishlist data is stored using browser localStorage.

Advantages:

- Fast access
- No authentication required
- Persistent across refresh

13 Deployment Architecture

13.1 Frontend Deployment

Frontend deployed on Vercel.

Advantages:

- Fast CDN delivery
- Next.js optimization
- Automatic deployment

13.2 Backend Deployment

Backend deployed on Render.

Advantages:

- Python support
- FastAPI compatibility
- Public API hosting

14 Deployment Links

Frontend:

<https://ai-fashion-agent-ecru.vercel.app/>

Backend:

<https://ai-fashion-agent.onrender.com>

GitHub Repository:

<https://github.com/Deepti25sahu/ai-fashion-agent>

15 Security Measures

- Environment variables for API keys
- CORS middleware configuration
- Hidden secret files using .gitignore
- Secure external links

16 Future Improvements

Future enhancements include:

- AI Chatbot Integration
- Virtual Try-On
- Firebase Authentication
- Payment Gateway
- Personalized Fashion Profiles
- Image-Based Fashion Search
- AI Trend Prediction

17 Conclusion

The AI Fashion Shopping Agent combines AI-driven recommendation systems, modern frontend engineering, conversational search, and premium ecommerce design to create an intelligent shopping assistant.

The project demonstrates scalable architecture, modular AI agents, and modern full-stack deployment practices suitable for real-world ecommerce platforms.